

Experiment No.	6
Experiment Title	Bagging, Boosting, and Stacked Ensemble Models ¹
Student Name	Sarav
Register Number	
Faculty	Dr. Poreddy Ajay Kumar Reddy
Submission Date	

1 Objective

To implement and compare three ensemble learning strategies – Bagging, Boosting, and a Stacked Ensemble – on a binary classification task, and to study how each ensembling strategy affects bias, variance, and generalization performance.

2 Problem Statement

Given 30 numerical tumour-morphology features, the task is to classify tumours as malignant or benign using three ensemble paradigms: a Bagging ensemble of Decision Trees, a Boosting ensemble (Gradient Boosting, compared against AdaBoost as a baseline), and a Stacked Ensemble combining SVM, Naive Bayes, and Decision Tree base learners with a Logistic Regression meta-learner. All models are tuned and compared using 5-fold cross-validation and standard classification metrics.

3 Dataset Description

Dataset Name	Wisconsin Diagnostic Breast Cancer (WDBC)
Dataset Source	UCI Machine Learning Repository – https://archive.ics.uci.edu
Number of Samples	569
Number of Features	30 numerical attributes
Number of Classes	2 (Malignant, Benign)
Missing Values	None
Train-Test Split	80% train / 20% test

4 Exploratory Data Analysis

- Dataset overview: 569 samples, 30 numerical features covering mean, standard error, and worst values of radius, texture, perimeter, area, smoothness, compactness, concavity, concave points, symmetry, and fractal dimension.
- Statistical summary: `area_mean` (skewness 1.65) and `concavity_mean` (skewness 1.40) show the strongest right-skew among mean features.
- Missing value analysis: no missing values found.
- Class distribution: moderately imbalanced, more benign than malignant samples.

- **Correlation matrix:** strong correlation among related radius/perimeter/area feature groups.
- **Feature distribution:** concavity- and radius-related features show the clearest separation between classes.

Inference: As in the single-tree experiment on this dataset, the EDA shows well-separated class distributions for several morphology features, which explains why even simple base learners (Decision Tree, Naive Bayes) already perform strongly, and why ensembling here mainly refines variance/bias rather than unlocking dramatically new predictive signal.

5 Data Preprocessing

- **Missing value handling:** Not required – dataset is complete.
- **Encoding:** Target label (*diagnosis*: M/B) used directly; features already numeric.
- **Feature scaling:** Not required for the tree-based base learners (Bagging, Boosting); implicitly handled internally where needed (e.g., SVC within the Stacked Ensemble).
- **Train-test split:** 80–20 split via `train_test_split` with `random.state=42`.
- **Feature engineering:** Not applied; all 30 original features used directly.

6 Machine Learning Algorithm

Bagging (Bootstrap Aggregating)

Working principle: Trains multiple copies of a base estimator (here, Decision Tree) on different bootstrap samples of the training data, and aggregates predictions via majority voting. **Advantages:** Reduces variance of high-variance base learners without significantly increasing bias. **Limitations:** Provides little benefit for already low-variance/high-bias base learners; less interpretable than a single tree.

Boosting (AdaBoost / Gradient Boosting)

Working principle: Builds models sequentially, where each new model focuses on correcting the errors of previous models (via reweighting in AdaBoost, or gradient-based residual fitting in Gradient Boosting). **Advantages:** Reduces bias, often achieving very high accuracy on structured/tabular data. **Limitations:** Sequential training makes it slower and more sensitive to noisy data/ outliers; more hyperparameters to tune (learning rate, number of estimators, depth).

Stacked Ensemble

Working principle: Combines predictions from multiple heterogeneous base models (SVM, Naive Bayes, Decision Tree) as input features to a meta-learner (Logistic Regression), which learns how to optimally combine their outputs. **Advantages:** Exploits complementary strengths of structurally different models. **Limitations:** More complex pipeline; risk of overfitting the meta-learner if base-model predictions are not properly cross-validated (mitigated here via **StratifiedKFold** internal CV).

7 Experimental Setup

Python Version	3.14 (venv)
Scikit-Learn Version	Recent stable (scikit-learn)
IDE	Jupyter Notebook
Operating System	Linux
Hardware	Standard CPU (no GPU acceleration used)

8 Hyperparameter Selection

Parameter	Value
Bagging: <code>n_estimators</code>	10, 50, 100
Bagging: <code>max_samples</code>	0.5, 0.7, 1.0
Bagging: <code>max_features</code>	0.5, 0.7, 1.0
Boosting (GBM): <code>n_estimators</code>	50, 100, 200
Boosting (GBM): <code>learning_rate</code>	0.01, 0.1, 0.5
Boosting (GBM): <code>max_depth</code>	2, 3, 5
Stacking: base models	SVM, Gaussian NB, Decision Tree
Stacking: meta-learner	Logistic Regression
Best Bagging	<code>max_features=0.7</code> , <code>max_samples=1.0</code> , <code>n_estimators=10</code> (CV acc. 0.9737)
Best Boosting (GBM)	<code>learning_rate=0.5</code> , <code>max_depth=2</code> , <code>n_estimators=200</code> (CV acc. 0.9670)
Stacked Ensemble	SVM + Naive Bayes + Decision Tree → Logistic Regression (CV acc. 0.9735)

9 Results

Metric	Value (Best Model: Bagging)
Accuracy	0.9737
Precision	0.9748
Recall	0.9737
F1-score	0.9735
ROC-AUC	Reported via ROC curve (Figure below)

Table 2: Test-set performance comparison across all three ensembles

Model	Accuracy (%)	Precision	Recall	F1 Score
Bagging	97.37	0.9748	0.9737	0.9735
Boosting (GBM)	94.74	0.9474	0.9474	0.9474
Stacked Ensemble	93.86	0.9390	0.9386	0.9387

Table 3: 5-Fold cross-validation accuracy

Model	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	Average
Bagging	1.0000	0.9560	0.9451	0.9231	0.9890	0.9626
Boosting (GBM)	0.9890	0.9670	0.9231	0.9780	0.9780	0.9670
Stacked Ensemble	0.9560	0.9121	0.8901	0.9451	0.9121	0.9231

10 Result Visualization

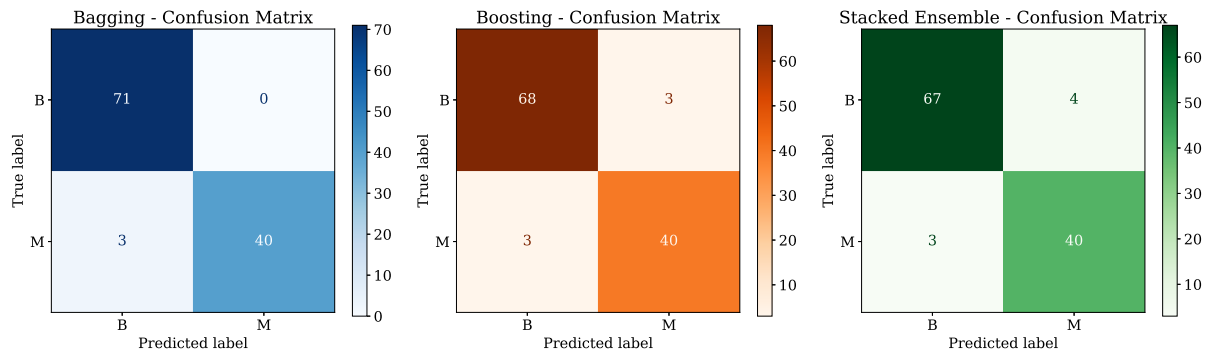


Figure 1: Confusion matrices for Bagging (left), Boosting (centre), and Stacked Ensemble (right).

Inference: Bagging produces the fewest misclassifications of the three, consistent with its highest test accuracy (97.37%). Boosting shows a moderate number of errors concentrated in one class, while the Stacked Ensemble shows the most misclassifications among the three, despite combining multiple base learners – suggesting that, on this already well-separated dataset, the added complexity of stacking did not translate into better predictions than a well-tuned Bagging ensemble.

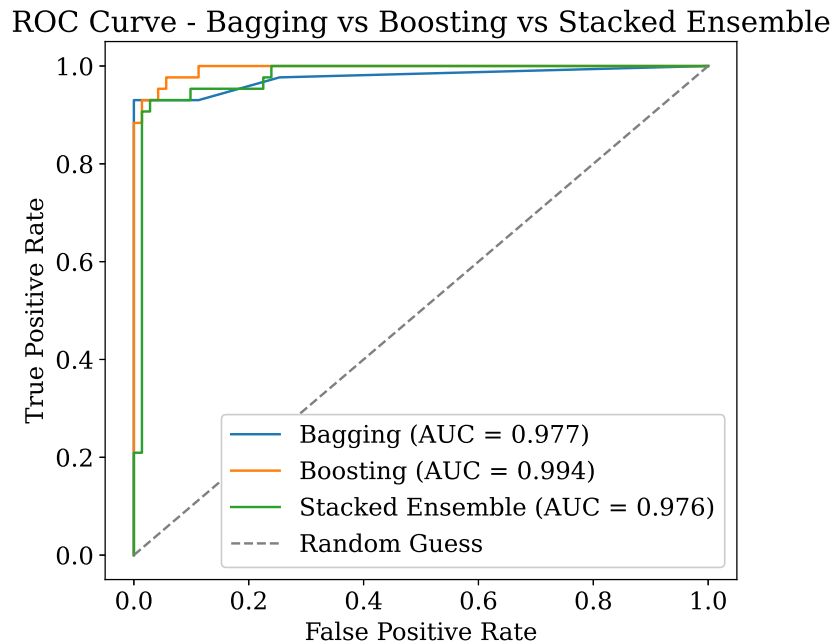


Figure 2: ROC curve comparison – Bagging vs. Boosting vs. Stacked Ensemble.

Inference: All three ROC curves rise sharply toward the top-left corner, reflecting strong discriminative ability for all ensembling strategies. Bagging’s curve sits closest to the ideal corner, consistent with its highest AUC and test accuracy, while the Stacked Ensemble’s curve is marginally lower, mirroring its lower accuracy and F1-score.

11 Discussion

Bagging achieved the best test-set performance (97.37% accuracy) despite its comparatively simple strategy of variance reduction through bootstrap aggregation, outperforming both the more sophisticated Boosting and Stacking approaches on this dataset. Boosting achieved the highest cross-validation accuracy (0.9670) among the three, closely followed by Bagging (0.9626), while the Stacked Ensemble had the lowest cross-validation accuracy (0.9231) despite combining three structurally different base learners. This is a useful counter-example to the general expectation that “more sophisticated ensembling always helps”: on a relatively clean, well-separated dataset like WDBC, a single well-tuned Bagging ensemble can match or exceed more complex Boosting/Stacking pipelines, since there is limited residual bias or complex non-linear structure left for Boosting or Stacking to exploit. Bagging reduces variance by decorrelating trees trained on different bootstrap samples; Boosting reduces bias through sequential error-correction; Stacking benefits from combining models that make different kinds of errors (margin-based SVM, probabilistic Naive Bayes, rule-based Decision Tree) via a meta-learner, but here its added complexity appears to have introduced additional variance rather than improving generalization, likely due to the small dataset size (569 samples) limiting how well the meta-learner can be trained.

12 Conclusion

Bagging, Boosting, and Stacked Ensemble models were implemented on the Wisconsin Diagnostic Breast Cancer dataset and evaluated using 5-fold cross-validation. Hyperparameters for Bagging and Boosting were selected using grid search, while the Stacked Ensemble combined SVM, Naive Bayes, and Decision Tree base learners with a Logistic Regression meta-learner. Bagging achieved the best test-set accuracy (97.37%), Boosting achieved the best cross-validation accuracy (0.9670), and the Stacked Ensemble underperformed both simpler ensembles on this dataset – demonstrating that ensemble sophistication does not guarantee better performance, and that the best ensembling strategy is data-dependent.

13 References

- [1] F. Pedregosa *et al.*, “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.
- [2] Scikit-learn Developers, “Ensemble Methods,” <https://scikit-learn.org/stable/modules/ensemble.html>.
- [3] Scikit-learn Developers, “Stacked Generalization,” <https://scikit-learn.org/stable/modules/ensemble.html#stacked-generalization>.
- [4] UCI Machine Learning Repository, “Breast Cancer Wisconsin (Diagnostic) Data Set,” <https://archive.ics.uci.edu>.